Code: Tái sử dụng mô hình trong Keras

```
# Tai mot mo hinh da huan luyen (Model A)
model_A = keras.models.load_model("my_model_A.h5")

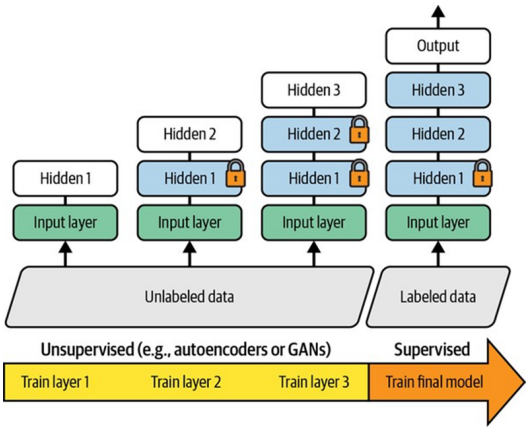
# Tao Model B dua tren cac lop cua Model A (tru lop cuoi)
model_B_on_A = keras.models.Sequential(model_A.layers[:-1])
model_B_on_A.add(keras.layers.Dense(1, activation="sigmoid"))

# Dong bang (Freeze) cac lop cu
for layer in model_B_on_A.layers[:-1]:
    layer.trainable = False

# Huan luyen vai epoch de cap nhat lop cuoi
model_B_on_A.compile(loss="binary_crossentropy", optimizer="sgd")
model_B_on_A.fit(X_train_B, y_train_B, epochs=4)
```

Huấn Luyện Không Giám Sát (Unsupervised Pretraining)

- Điều gì xảy ra nếu bạn có quá ít dữ liệu gắn nhãn (Labeled data)?
- Bạn có thể đào tạo trước bằng Học không giám sát (như Autoencoders hoặc RBMs).
- Sau đó chèn thêm lớp phân loại và huấn luyện có giám sát.
- **Hình 11-6**: Các mô hình NLP ngôn ngữ lớn như GPT và BERT cũng là một dạng Unsupervised Pretraining khổng lồ!



Hình 11-6: Học chuyển giao từ huấn luyện không giám sát

Giới thiệu các Optimizer (Trình tối ưu hóa)

- SGD (Stochastic Gradient Descent) cơ bản hoạt động ổn định nhưng **cực kỳ chậm**.
- Các nhà khoa học đã phát triển ra các thuật toán Tối ưu hóa có khả năng chạy nhanh gấp 10-100 lần.
- Các Optimizer nổi tiếng:
  1. Momentum Optimization
  2. Nesterov Accelerated Gradient (NAG)
  3. AdaGrad
  4. RMSProp
  5. Adam / Nadam

Tối ưu hóa Bằng Động Lượng (Momentum)

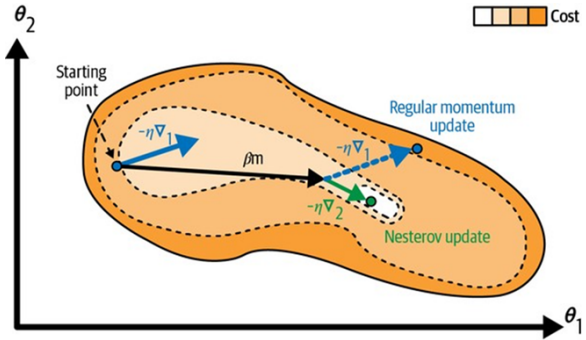
- SGD cập nhật trọng số dựa hoàn toàn trên **gradient tại thời điểm hiện tại**. Nó giống như đi xuống núi nhưng rẽ ngoặt theo góc dốc ngay dưới chân.
- Momentum quan tâm đến **các gradient trước đó**. Giống như thả một quả bóng bowling xuống bề mặt trơn: quả bóng sẽ liên tục **tăng tốc** (động lượng lớn).
- Cập nhật công thức:

$$m \leftarrow \beta m - \eta \nabla_\theta J(\theta)$$
$$\theta \leftarrow \theta + m$$

- Keras: `keras.optimizers.SGD(learning_rate=0.001, momentum=0.9)`

Nesterov Accelerated Gradient (NAG)

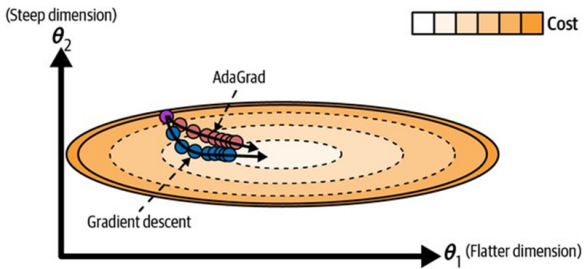
- Biến thể của Momentum.
- Thay vì tính đạo hàm tại vị trí hiện tại θ, NAG tính đạo hàm ở bước **tiến tới (Look-ahead)** một khoảng βm.
- Điều này giúp thuật toán "đạp phanh"kịp thời khi đến gần đáy lũng.
- **Hình 11-7** so sánh Momentum thông thường (màu xanh lam) và NAG (màu đỏ).



Hình 11-7: So sánh Momentum thông thường và Nesterov

AdaGrad

- Trong các bài toán có nhiều chiều (dimensions) bị thon dài (elongated bowl).
- Thuật toán AdaGrad tự động **làm giảm tốc độ học** dọc theo các chiều có độ dốc quá cao (steep dimensions).
- **Hình 11-8**: AdaGrad đi thẳng tới đích rành mạch hơn thay vì "ngoằn ngoèo"như SGD.
- Lưu ý: AdaGrad bị dừng quá sớm khi dùng cho Neural Networks, do vậy chỉ dùng trong hồi quy tuyến tính.



Hình 11-8: So sánh AdaGrad và Gradient Descent

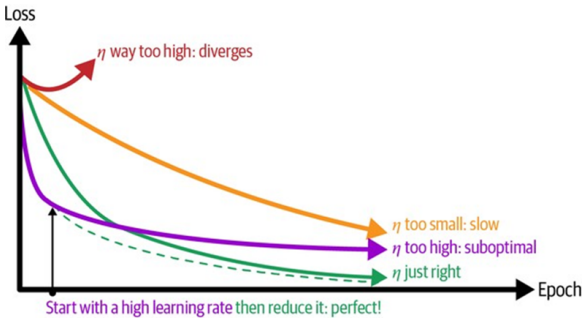
RMSProp và Adam

- **RMSProp**: Sửa chữa nhược điểm của AdaGrad bằng cách chỉ xem xét các gradient gần đây (thay vì tích lũy từ đầu). Rất phổ biến trước khi Adam ra đời.
- **Adam (Adaptive Moment Estimation)**: Kết hợp tính chất tuyệt vời của Momentum (tích lũy gradient) và RMSProp (tích lũy bình phương gradient).
- Adam là Optimizer "mặc định"và thành công vang dội nhất hiện nay cho Mạng Nơ-ron.

```
# Hien tai Optimizer xin so va de dung nhat la Adam
optimizer = keras.optimizers.Adam(learning_rate=0.001, beta_1=0.9, beta_2=0.999)
```

Vấn đề của Tốc Độ Học Cố Định

- Tốc độ học (Learning Rate - η) là siêu tham số quan trọng nhất.
- Quá lớn: mô hình phân kỳ. Quá nhỏ: mất quá nhiều thời gian hội tụ.
- **Hình 11-9**: Nếu ta bắt đầu lớn (để tiến nhanh), sau đó giảm dần (để hội tụ chính xác), ta sẽ đạt kết quả tối ưu nhất trong thời gian ngắn nhất!



Hình 11-9: Lịch trình tốc độ học

Code: Power Scheduling trong Keras

- Lập lịch theo hàm lũy thừa: $\eta(t) = \eta_0/(1 + t/s)^c$.
- Keras thực hiện cực kì dễ thông qua tham số 'decay' (Keras phiên bản cũ) hoặc 'InverseTimeDecay'.

```
lr_schedule = keras.optimizers.schedules.InverseTimeDecay(
    initial_learning_rate=0.01,
    decay_steps=10000,
    decay_rate=1.0)

optimizer = keras.optimizers.SGD(learning_rate=lr_schedule)
```

Code: Performance Scheduling

- Giảm tốc độ học một nửa khi hàm lỗi (validation error) ngưng giảm.
- Trong Keras, sử dụng Callback `ReduceLROnPlateau`.

```
lr_scheduler = keras.callbacks.ReduceLROnPlateau(
    factor=0.5, # giam mot nua
    patience=5) # neu sau 5 epoch khong giam loi, thi giam LR

model.fit(X_train, y_train, epochs=50,
          validation_data=(X_valid, y_valid),
          callbacks=[lr_scheduler])
```

Tổng Quan Các Phương Pháp Chính Quy Hóa

- "Với hàng tỷ trọng số tự do, mạng nơ-ron sâu có thể học rất tốt, thậm chí nhớ luôn cả nhiễu trong dữ liệu."
- Đây chính là **Overfitting** (Quá khớp).
- Các kỹ thuật phổ biến để khắc phục:
  1. L1 và L2 Regularization (Kìm hãm trọng số lớn)
  2. Dropout (Loại bỏ ngẫu nhiên)
  3. Max-Norm (Giới hạn chuẩn vector trọng số)
  4. Data Augmentation (Tăng cường dữ liệu bằng biến đổi hình ảnh/âm thanh)

L1 và L2 Regularization

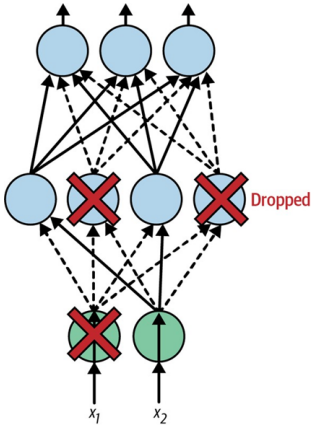
- Cộng thêm hình phạt (penalty) bằng độ lớn của trọng số (Weights) vào hàm Mất Mát (Loss Function).
- Có thể thiết lập dễ dàng trong Keras layers:

```
# Ap dung l2 regularization voi he so 0.01
layer = keras.layers.Dense(100, activation="elu",
                           kernel_initializer="he_normal",
                           kernel_regularizer=keras.regularizers.l2(0.01))
```

- Để mã sạch hơn (do lặp lại nhiều lần), bạn có thể dùng tính năng 'functools.partial'.

Dropout - "Liều Thuốc Mê"Cho Nơ-ron

- G. E. Hinton (2012) giới thiệu thuật toán đơn giản một cách vô lý nhưng cực kỳ thành công: **Dropout**.
- Tại mỗi lượt huấn luyện, mỗi nơ-ron có xác suất p (thường từ 10% đến 50%) bị "ngắt"tạm thời.
- Các nơ-ron không thể phụ thuộc vào đồng nghiệp kế bên mà tự thân vận động trích xuất các đặc trưng hữu ích nhất.
- **Hình 11-10**: Minh họa mạng áp dụng kỹ thuật Dropout.



Hình 11-10: Với chính quy hóa dropout, một số nơ-ron bị tắt ngẫu nhiên

Sử dụng Dropout trong Keras

- **Quan trọng**: Khi Dropout huấn luyện, chỉ còn khoảng một nửa số nơ-ron tham gia dự đoán. Vì vậy, trong giai đoạn **Test/Inference**, ta PHẢI nhân giá trị với xác suất giữ lại để cân bằng. (Keras tự động xử lý việc này).

```
model = keras.models.Sequential([
    keras.layers.Flatten(input_shape=[28, 28]),
    keras.layers.Dropout(rate=0.2), # ngat 20% no-ron
    keras.layers.Dense(300, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(100, activation="elu", kernel_initializer="he_normal"),
    keras.layers.Dropout(rate=0.2),
    keras.layers.Dense(10, activation="softmax")
])
```

Monte Carlo (MC) Dropout

- Y. Gal và Z. Ghahramani (2016) chỉ ra rằng mạng nơ-ron sử dụng Dropout tương đương toán học với quá trình xấp xỉ suy luận Bayes (Bayesian inference).
- Cụ thể, **MC Dropout** bật Dropout ngay cả trong giai đoạn Inference (Dự đoán).
- Khi ta thực hiện dự đoán 100 lần trên cùng 1 mẫu, ta sẽ nhận được 100 kết quả dự đoán (xác suất) khác nhau do mỗi lần mạng lại tắt đi một số nơ-ron ngẫu nhiên.
- Tính trung bình và phương sai của các dự đoán đó sẽ cho ta một độ tin cậy vô cùng chính xác!